Analyse Control Flow, String Operations and Functions in Python

An Assignment REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0807 Python Programming

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

Muthupandi C (192565037)

Under the Supervision of

Dr. Haripriya

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

OCTOBER 2026

GITHUB LINK: muthupandic84-creator/CSA0805--192565037--Python-Program: CSA0805

CONTENTS

- Scenario Background – Python Programming Assignment 2
- Q1 – Decisions with Many Boundaries
- Q2 – Loops that Count Attempts
- Q3 – Engineer a Vehicle Registration Number Validator
- Q4 – Building Long Text: + or join()?
- Q5 – Functions, Parameters and Scope
- Q6 – Recursion and Its Limits
- Q7 – Diagnose, Redesign and Validate

Complexity Summary

Conclusion and Execution Evidence

Scenario Background – Python Programming Assignment 2

This assignment studies practical Python programming problems drawn from public-service decisions, bank kiosk OTP verification, vehicle registration records, SMS statement generation, canteen order processing, and counting stair-climbing combinations. The common objective is to choose suitable control-flow and data-handling structures, implement them correctly, and support design choices with test evidence and Big-O analysis.

The work follows the required sequence: predict, design, implement, measure, analyse, evaluate, diagnose, redesign, and validate. Each task is stored in a separate folder so that programs can be executed and checked independently.

Complexity and Design Note

Q1 – Decisions with Many Boundaries

The electricity subsidy policy has three mutually exclusive consumption bands. A household using 100 units or fewer receives full subsidy. A household using 101 through 200 units receives partial subsidy only when it has a ration card. Above 200 units, subsidy is available only when a member is older than 80.

Before execution, the predicted cases are: 99 and 100 units receive full subsidy; 101 and 200 units depend on ration-card status; 201 units without an elderly member receives no subsidy; and a household above 200 with an elderly member receives the special exemption. The implementation compares independent if statements with an if–elif–else chain.

Separate if statements are evaluated independently, so overlapping conditions can append more than one result. An if–elif–else chain selects only the first matching branch. For the stated non-overlapping bands both designs can agree, but the chain expresses the exclusive policy more clearly. A nested test keeps the ration-card or age condition beside the relevant band.

Alternative concept: match–case can use guarded cases such as `case u if u <= 100`. A list of upper limits and decisions can be scanned in order. With k slabs, a linear scan takes O(k) time in the worst case; both approaches use O(1) auxiliary space apart from the rule table.

Execution Output

```
ID  Units  Ration  >80  Separate-if result              Chained result
H1  99     False   False Full subsidy                   Full subsidy
H2  100    True    False Full subsidy                   Full subsidy
H3  101    True    False Partial subsidy                Partial subsidy
H4  101    False   False No subsidy                     No subsidy
H5  150    True    False Partial subsidy                Partial subsidy
H6  200    False   True No subsidy                      No subsidy
H7  201    False   False No subsidy                     No subsidy
H8  250    False   True Special elderly exemption       Special elderly exemption

Boundary checks (100, 101, 200, 201):
100 => Full subsidy
101 => Partial subsidy
200 => Partial subsidy
201 => Special elderly exemption

Design decision: use if-elif-else for mutually exclusive unit bands.
Nested checks keep the eligibility conditions close to each band.
```

Complexity and Design Note

Decision evaluation is bounded by the number of branches: O(k) time and O(1) auxiliary space.

Q2 – Loops that Count Attempts

The kiosk permits three non-blank OTP attempts. A blank entry is ignored. A correct OTP exits early with break; continue skips blank input; pass is a no-operation placeholder. A loop else clause runs when the loop finishes without break and can report a lock after all permitted attempts are consumed.

A while loop is natural when the attempt limit changes after a security-question check or when blank inputs must not consume attempts. A for loop over range(3) is concise for a fixed number of attempts, but a blank input requires care: the loop index and the number of counted attempts are not the same quantity. Updating the attempt counter before checking for blank input causes a blank to consume an attempt; failing to update it can cause an infinite loop.

When the security answer is correct, the maximum can be changed to five. A while loop handles this dynamic limit directly. A recursive design calls itself with the next input index and updated attempt count. All three approaches take O(a) time for a attempts. A loop uses O(1) auxiliary space; recursion uses O(a) call-stack space.

Execution Output

```
WHILE DEMO:
Blank entry: not counted
Wrong OTP (1/3)
Wrong OTP (2/3)
OTP verified using while loop
\nFOR RANGE(3) DEMO:
Wrong OTP (1/3)
Wrong OTP (2/3)
Wrong OTP (3/3)
Account locked for 30 minutes
\nRECURSIVE DEMO: Account locked for 30 minutes
\nChanged condition: after successful security answer, call with max_attempts=5.
A while loop handles skipped blank entries and dynamically changed limits more naturally.
```

Complexity and Design Note

For a attempts, loop and recursive solutions take O(a) time; loops use O(1) extra space and recursion uses O(a) stack space.

Q3 – Engineer a Vehicle Registration Number Validator

The accepted format is two state letters, two district digits, one or two series letters, and four final digits. The normalise function strips outer whitespace, removes spaces and hyphens, and converts to uppercase. Validation checks total length and each slice using isalpha() and isdigit(), without regular expressions. The receipt formatter inserts spaces between the four components.

Strings are immutable: upper() returns a new string and does not modify the original object. For a 10-character registration, s[-4:] returns the final four digits and s[4:-4] returns the two-letter series. For a 9-character registration, those slices return the final four digits and one-letter series.

Alternative concept: `re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg)` verifies the complete string. Both approaches are O(L) in the length L. Slicing exposes each field clearly; a regular expression is concise but may be less approachable for a beginner.

Execution Output

```
Input             Normalised   Result   Receipt
tn 09 ab 1234       TN09AB1234   valid    TN 09 AB 1234
KA-01-M-0007        KA01M0007    valid    KA 01 M 0007
TN9AB1234           TN9AB1234    INVALID  -
TN 22 1234 AB       TN221234AB   INVALID  -
mh 12 xyz 6789      MH12XYZ6789  INVALID  -
DL-8C-1234          DL8C1234     INVALID  -

Slice inquiry:
TN09AB1234 length = 10 last four = 1234 middle = AB
TN09A1234 length = 9 last four = 1234 middle = A

Strings are immutable: upper() returns a new string; it does not alter the original.
```

Complexity and Design Note

Normalisation and validation inspect characters a constant number of times: O(L) time; the normalised output uses O(L) space.

Q4 – Building Long Text: + or join()?

The program creates the same monthly statement using repeated +=, a join operation, and f-string formatting for each line. It measures 1,000, 10,000, and 200,000 entries and repeats each measurement three times. The program prints individual measurements and averages so results can be recorded on the student's own computer.

Python strings are immutable. Repeated concatenation can repeatedly allocate strings and copy previous characters, resulting in up to $O(n^2)$ character-copying work in an unfavourable implementation. join() can assemble the output in O(n) time after determining the parts. CPython can optimise some local += patterns, so measured results may vary and should not be fabricated or assumed.

Alternative concept: io.StringIO provides an in-memory text buffer and supports incremental writes. Writing each line directly to a file avoids keeping the entire final statement in RAM, which is useful for very large reports. join() is convenient when a final string is required; file streaming is preferable when output can be written directly.

Execution Output

```
Lines: 1,000 | same output: True
+=      samples=[9.4e-05, 0.000161, 8.6e-05] average=0.000114s
join    samples=[8.9e-05, 7.4e-05, 6.9e-05] average=0.000078s
f-string samples=[0.000108, 0.000173, 6.7e-05] average=0.000116s

Lines: 10,000 | same output: True
+=      samples=[0.001072, 0.001031, 0.000905] average=0.001003s
join    samples=[0.001175, 0.000828, 0.000664] average=0.000889s
f-string samples=[0.000678, 0.000611, 0.000552] average=0.000614s

Lines: 200,000 | same output: True
+=      samples=[0.021398, 0.021011, 0.024905] average=0.022438s
join    samples=[0.018543, 0.018746, 0.018508] average=0.018599s
f-string samples=[0.015278, 0.01381, 0.012403] average=0.013831s
```

Complexity and Design Note

join is O(n) in output size; repeated concatenation may approach $O(n^2)$ copying. The final output requires O(n) memory.

Q5 – Functions, Parameters and Scope

A mutable default parameter such as order=[] is created once when the function is defined. Calls that omit the argument therefore share the same list, allowing one customer's items to appear in another customer's order. The safe pattern uses order=None and creates a fresh list inside the function.

Positional parameters are useful when argument order is obvious; keyword parameters improve readability; default parameters support common values. A function can return more than one value as a tuple, as shown by the total and GST calculation. Local names exist within a function. Global names are defined at module level and can be read inside functions. Rebinding a global name requires the global keyword. A nested function can rebind an enclosing function's variable using nonlocal.

Global mutation is usually avoided because it creates hidden dependencies and complicates testing. The closure counter demonstrates nonlocal scope without changing a module-level variable.

Execution Output

```
Buggy shared default:
['Tea', 'Vada'] ['Tea', 'Vada']
Correct independent defaults:
['Tea'] ['Vada']
Three customer orders: [['Tea'], ['Vada'], ['Coffee']]
Total including GST: Rs 94.50; GST: Rs 4.50
This is local
Global value: SIMATS ENGINEERING
Nonlocal counter: 1 2 3
Buggy default list object: (['Tea', 'Vada'],)
```

Complexity and Design Note

Appending one item to a list is amortised O(1). Each independent customer's order requires O(m) storage for m items.

Q6 – Recursion and Its Limits

The number of ways to climb n steps using one- or two-step moves follows ways(n) = ways(n-1) + ways(n-2), with base cases ways(0) = ways(1) = 1. The values for n=1 through 6 are 1, 2, 3, 5, 8, and 13.

Plain recursion repeats the same subproblems. For n=5, the call tree branches to n=4 and n=3; both branches later calculate n=3 or n=2 again. The naive implementation therefore has exponential time, $O(\varphi^n)$, and O(n) stack space. Memoisation stores each computed value, reducing time to O(n), with O(n) cache and recursion-stack space. The iterative version stores only the previous two values, using O(n) time and O(1) auxiliary space.

A memoised recursive call at n=1000 may exceed Python's recursion limit and raise RecursionError. The deployment recommendation is to use the iterative method for this input size rather than relying on a raised recursion limit, which can risk exhausting the interpreter's stack.

Execution Output

```
Hand-check values:
n=1: 1
n=2: 2
n=3: 3
n=4: 5
n=5: 8
n=6: 13

Call counts:
n=10: naive=89, calls=0; memo=89, calls=0
n=20: naive=10946, calls=0; memo=10946, calls=0
n=25: naive=121393, calls=0; memo=121393, calls=0
n=30: naive=1346269, calls=0; memo=1346269, calls=0

For n=1000:
memoised result:
70330367711422815821835254877183549770181269836358732742604905087154537118196933579742249494562611733487750449241765991088
18636326545022364710601205337412127386733911119813937312559876769009190224524532340350 1
Loop result for n=1000:
70330367711422815821835254877183549770181269836358732742604905087154537118196933579742249494562611733487750449241765991088
18636326545022364710601205337412127386733911119813937312559876769009190224524532340350 1
The loop avoids recursion depth and uses O(1) extra space.
```

Complexity and Design Note

Naive recursion $O(\varphi^n)$, memoisation O(n) time/O(n) space, iterative O(n) time/O(1) auxiliary space.

Q7 – Diagnose, Redesign and Validate

Selected limitation: plain recursive stair counting in Q6. The bottleneck is repeated evaluation of identical subproblems. The number of recursive calls grows exponentially with n, even though only n distinct results are required.

Measurable objective: preserve the same answer while reducing time to O(n), avoiding recursive stack growth, and using O(1) auxiliary space. The redesign maintains two values: the number of ways for the previous step and the current step. Each iteration computes their sum and shifts the pair. This property removes repeated work and does not require a cache.

Prediction: the iterative design will return the same values for all tested n, run faster as n grows, and complete n=1000 without a recursion-depth error. The Task07 program compares original and redesigned outputs and timings for n=10, 20, 25, and 30, then runs the iterative method for n=1000. Timings must be rerun locally because they depend on the machine and Python build.

Evaluation: if outputs match and n=1000 completes, the objective is achieved. The original recursive version remains useful for explaining the recurrence and for very small inputs, but the iterative design is safer for larger values.

Execution Output

```
Q7 redesign: naive recursive stair count -> iterative two-value update
Objective: preserve the answer while reducing repeated calls and stack use.
n=10: same=True, naive=0.000008s, loop=0.000002s
n=20: same=True, naive=0.000657s, loop=0.000002s
n=25: same=True, naive=0.007791s, loop=0.000003s
n=30: same=True, naive=0.100161s, loop=0.000006s
n=1000: result has 209 digits; loop time=0.000053s
Trade-off: iterative version is less directly recursive but uses O(1) extra space.
```

Complexity and Design Note

The iterative redesign is O(n) time and O(1) auxiliary space, avoiding repeated calls and recursion depth.

Complexity Summary

Approach	Time	Auxiliary Space
Q1 decision chain	O(k)	O(1)
Q2 loop	O(a)	O(1)
Q2 recursion	O(a)	O(a) stack
Q3 string processing	O(L)	O(L)
Q4 join	O(n)	O(n) output
Q4 repeated concatenation	Up to $O(n^2)$	O(n) output
Q5 append one item	O(1) amortised	O(1) extra
Q6 naive recursion	$O(\varphi^n)$	O(n) stack
Q6 memoisation	O(n)	O(n) cache + stack
Q6 iterative	O(n)	O(1)
Q7 redesign	O(n)	O(1)

Conclusion

The assignment demonstrates how choosing the correct control structure and data-handling approach improves correctness, readability, and performance. Testing boundary inputs and measuring execution time are important because theoretical complexity and actual timings describe different aspects of program behaviour.

Submission Evidence Checklist

- Run every Task01–Task07 program from the extracted project folder.
- Capture genuine terminal screenshots and place them in the matching task folders.
- Rerun the benchmark programs and retain the raw timing outputs.
- Create the GitHub repository and replace the URL placeholder on the cover page.
- Commit progressively, with at least one commit per task.

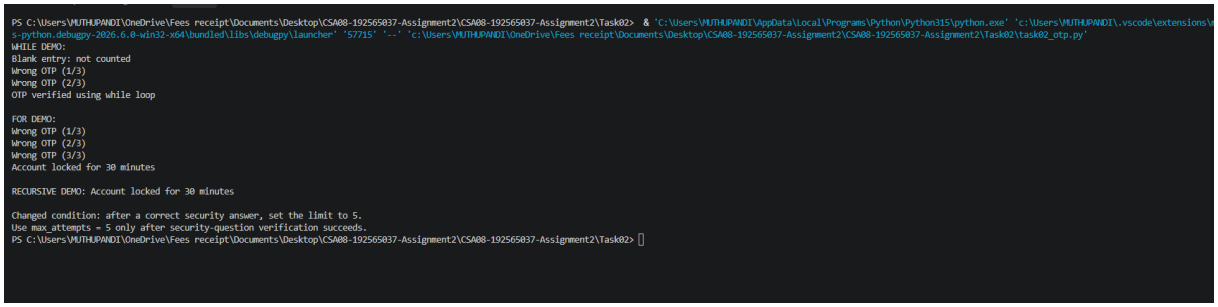
APPENDIX – ACTUAL PYTHON EXECUTION SCREENSHOTS

The following screenshots were supplied from the student's VS Code terminal. They provide execution evidence for the tasks indicated in each caption. Q4 and Q6 screenshots are not included in this upload and should be added after those programs are run.

Q1 – Subsidy decision output



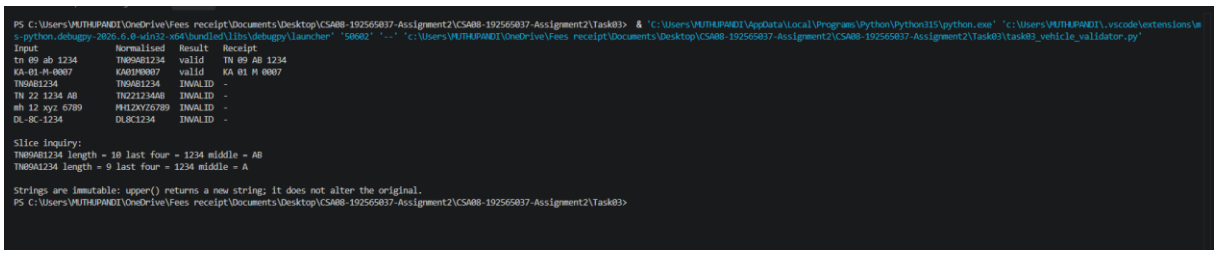
Q1 – Boundary and chained decision output



Q2 – OTP loop output



Q2 – OTP loop and lockout output



Q3 – Vehicle registration validation output

```
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task05> & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51770' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task05\task05_functions_scope.py'
Buggy shared default:
['Tea', 'Vada'] ['Tea', 'Vada']
Correct independent defaults:
['Tea'] ['Vada']
Three customer orders: [['Tea'], ['Vada'], ['Coffee']]
Total including GST: Rs 94.50; GST: Rs 4.50
This is local
Global value: SIMATS ENGINEERING
Nonlocal counter: 1 2 3
Buggy default list object: (['Tea', 'Vada'],)
```

Q5 – Functions, parameters and scope output

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task05> & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51770' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task05\task05_functions_scope.py'
Buggy shared default:
['Tea', 'Vada'] ['Tea', 'Vada']
Correct independent defaults:
['Tea'] ['Vada']
Three customer orders: [['Tea'], ['Vada'], ['Coffee']]
Total including GST: Rs 94.50; GST: Rs 4.50
This is local
Global value: SIMATS ENGINEERING
Nonlocal counter: 1 2 3
Buggy default list object: (['Tea', 'Vada'],)
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task05>
```

Q7 – Redesign benchmark output

```
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task07> & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51427' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task07\task07_redesign.py'
Q7 redesign: naive recursive stair count -> iterative two-value update
Objective: preserve the answer while reducing repeated calls and stack use.
n=10: same=True, naive=0.000016s, loop=0.000004s
n=20: same=True, naive=0.000976s, loop=0.000004s
n=25: same=True, naive=0.012561s, loop=0.000007s
n=30: same=True, naive=0.131019s, loop=0.000006s
n=1000: result has 209 digits; loop time=0.000101s
Trade-off: iterative version is less directly recursive but uses O(1) extra space.
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task07>
```

Additional terminal execution evidence

```
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task01> & 'C:\Users\MUTHUPANDI\AppData\Local\Programs\Python\Python315\python.exe' 'c:\Users\MUTHUPANDI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50978' '--' 'c:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task01\task01_subsidy.py'
ID  Units  Ration  >80  Separate-if result                  Chained result
H1  99     False   False Full subsidy                       Full subsidy
H2  100    True    False Full subsidy                       Full subsidy
H3  101    True    False Partial subsidy                    Partial subsidy
H4  101    False   False No subsidy                         No subsidy
H5  150    True    False Partial subsidy                    Partial subsidy
H6  200    False   True No subsidy                         No subsidy
H7  201    False   False No subsidy                         No subsidy
H8  250    False   True Special elderly exemption           Special elderly exemption

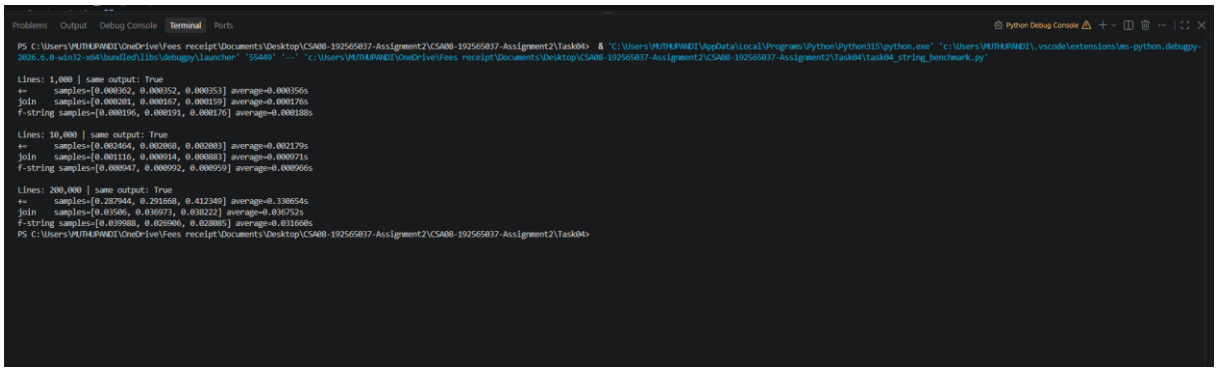
Boundary checks (100, 101, 200, 201):
100 => Full subsidy
101 => Partial subsidy
200 => Partial subsidy
201 => Special elderly exemption

Design decision: use if-elif-else for mutually exclusive unit bands.
Nested checks keep the eligibility conditions close to each band.
PS C:\Users\MUTHUPANDI\OneDrive\Fees receipt\Documents\Desktop\CSA08-192565037-Assignment2\CSA08-192565037-Assignment2\Task01>
```

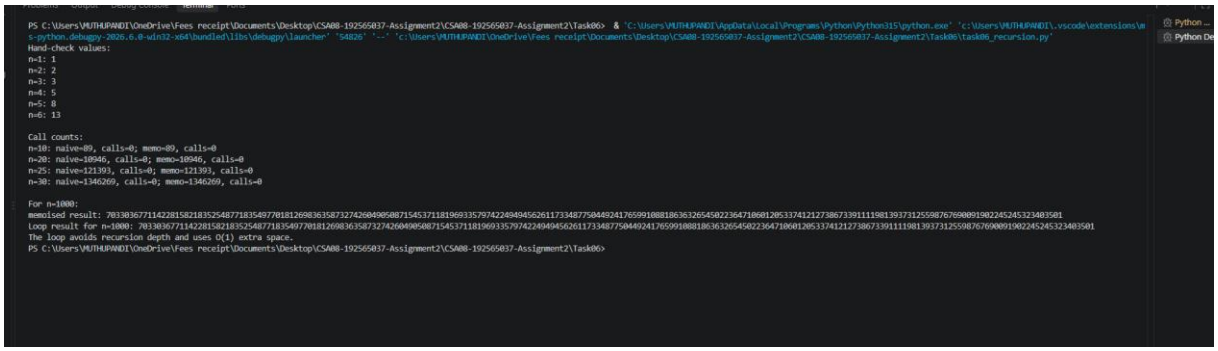
ADDITIONAL EXECUTION EVIDENCE – Q4 AND Q6

The following screenshots were supplied from the student's VS Code terminal and complete the Q4 and Q6 screenshot evidence.

Q4 – String building benchmark: 1,000, 10,000 and 200,000 lines



Q6 – Staircase recursion, memoisation and n=1,000 output



Q6 verification note

The supplied Q6 screenshot shows correct staircase values and matching memoised/iterative results for n=1,000. However, its displayed call counters are zero because the main function's counter reset was local rather than global. The Task06 source file in this updated ZIP fixes this by declaring `global calls` inside main(). Run the corrected file again to capture a replacement call-count screenshot; the supplied screenshot is retained as original evidence.